CSCI E-25

Document Image Classification - Multiclass classifier

Kamal Dalal

Introduction

Content-based analysis of document images has a number of applications. The economic feasibility of creating a large database of document images has left a tremendous need for robust ways to access the information. Printed documents are scanned for archiving or in an attempt to move towards a paperless office and are stored as images. In digital libraries, documents are often stored as images before they are processed by text extraction tools. Documents play a pivotal role in all of the fields of business communication and record-keeping.

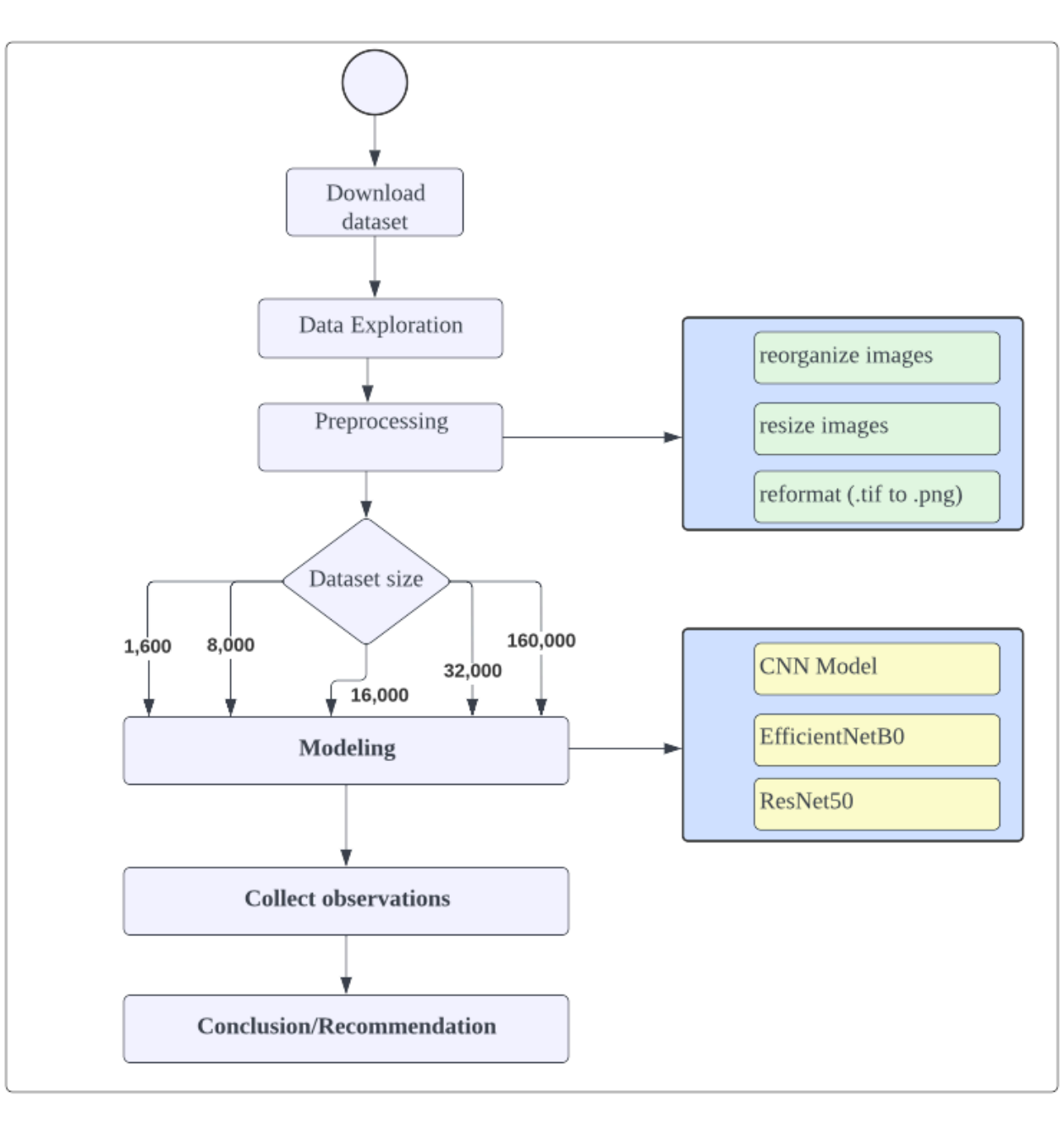
Document classification is an age-old problem in information retrieval, and it plays an important role in a variety of applications for effectively managing text and large volumes of unstructured information. Automatic document classification can be defined as content-based assignment of one or more predefined categories (topics) to documents. This makes it easier to find the relevant information at the right time and for filtering and routing documents directly to users. Automatic document classification applies machine learning or other technologies to automatically classify documents; this results in faster, scalable and more objective classification.

Objectivve

Build a model to classify input document image as one of predefined categories.

For this project, I am going to use supervised method, in which the classifier is trained on a prelabeled train document images. The classifier should predict a category for a document image. I will use publicly available dataset to train, and test the model.

Structure of the notebook:



As shown in the image, I will start with publicly available dataset.I will explore the dataset, perform pre-processing steps before designing models. Due to computation and memory limitation, I will create small datasets to build and validate models.

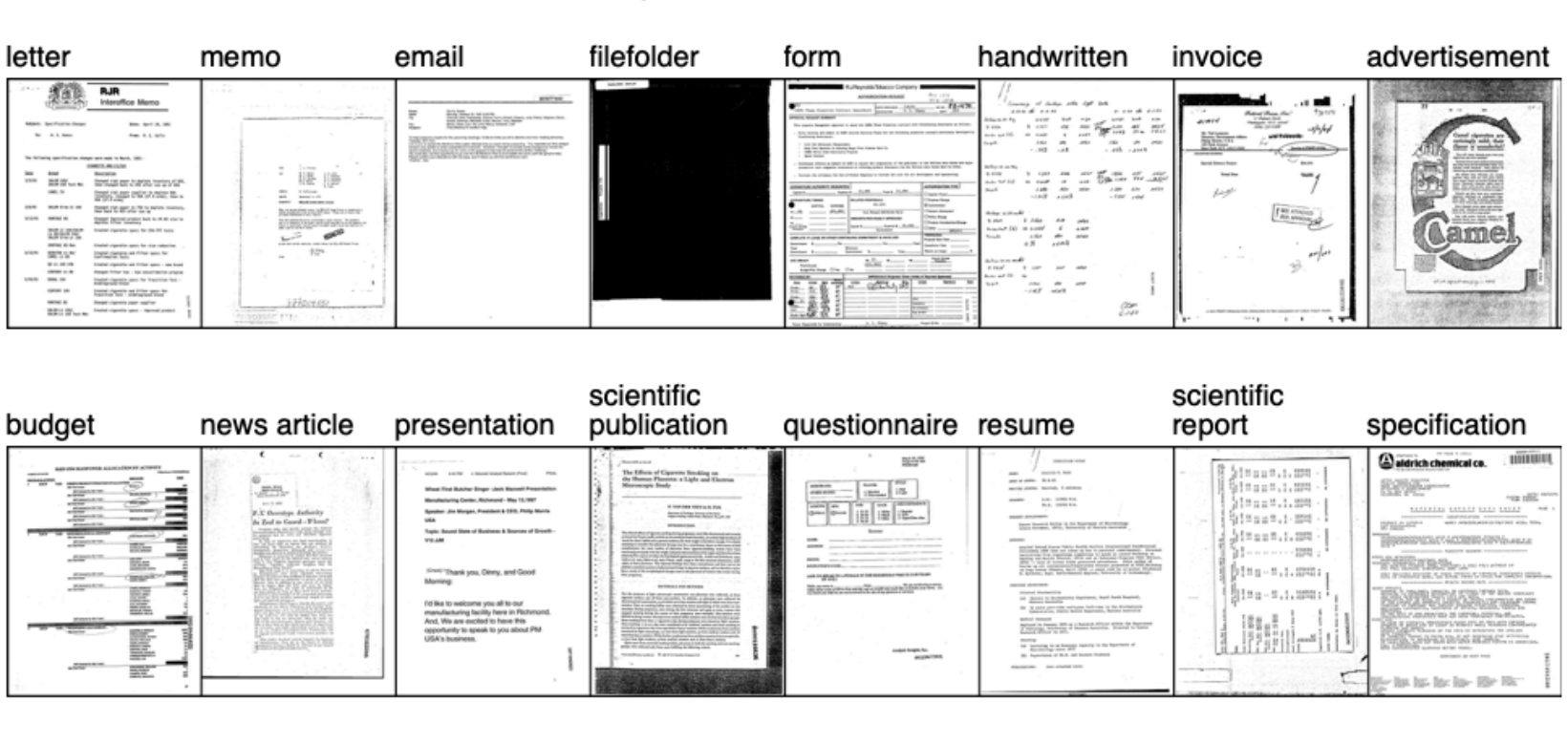
> I will build three different models:
>
> > 1. Convolution Neural Network
> > 2. Transfer learning using EfficientNetB0
> > 3. Transfer learning using ResNet50
>
> These three models will be executed against datasets with different number of train/test images. I will collect metrics like train accuracy, validation accuracy, train loss, validation loss, execution time, and number of epochs etc. after each run. At the end, I will conclude the project based on metrics data of these models.
>
> **Note:** Every dataset will have 80/20 (percent) train/test ratio images. Datasets will be labled based on train images size. Each dataset will contain 20% of train images for validation. These images will be in addition to train images.

Download dataset

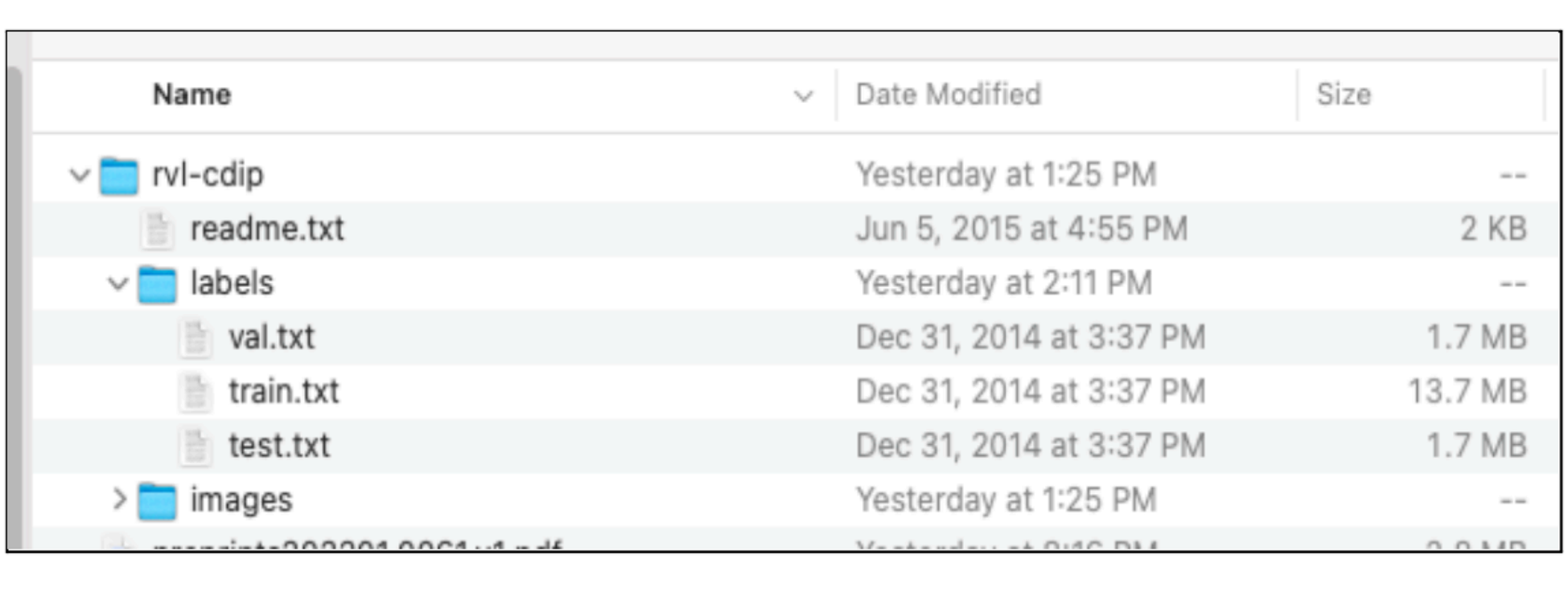
> For this project, I have used RVL-CDIP (Ryerson Vision Lab Complex Document Information Processing) dataset.The RVL-CDIP dataset consists of scanned document images belonging to 16 classes such as letter, form, email, resume, memo, etc. Dataset is publicly available at rvl-cdip.tar.gz, (approximate size 37GB compressed). The data set consists of scanned grayscale images of documents and is segregated into 16 categories or classes. The dataset is subdivided into Training, Validation and Test Sets each containing 320000, 40000 and 40000 images respectively. The dataset is already split into train/dev/test, with 320k/40k/40k files available in each split respectively. Uncompressed, the dataset size is ~100GB
>
> **Note:** These images are grayscaled.
>
> Ref: A. W. Harley, A. Ufkes, K. G. Derpanis, "Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval," in ICDAR, 2015 https://www.cs.cmu.edu/~aharley/icdar15

Data Exploration

> At this step, we will explore the data to understand about given images. There are 320,000 training images, 40,000 validation images, and 40,000 test images. The images are sized so their largest dimension does not exceed 1000 pixels. The label files list the images and their categories in the following format: path/to/the/image.tif category where the categories are numbered 0 to 15, in the following order:
>
> 1. letter
> 2. form
> 3. email
> 4. handwritten
> 5. advertisement
> 6. scientific report
> 7. scientific publication
> 8. specification
> 9. file folder
> 10. news article
> 11. budget
> 12. invoice
> 13. presentation
> 14. questionnaire
> 15. resume
> 16. memo



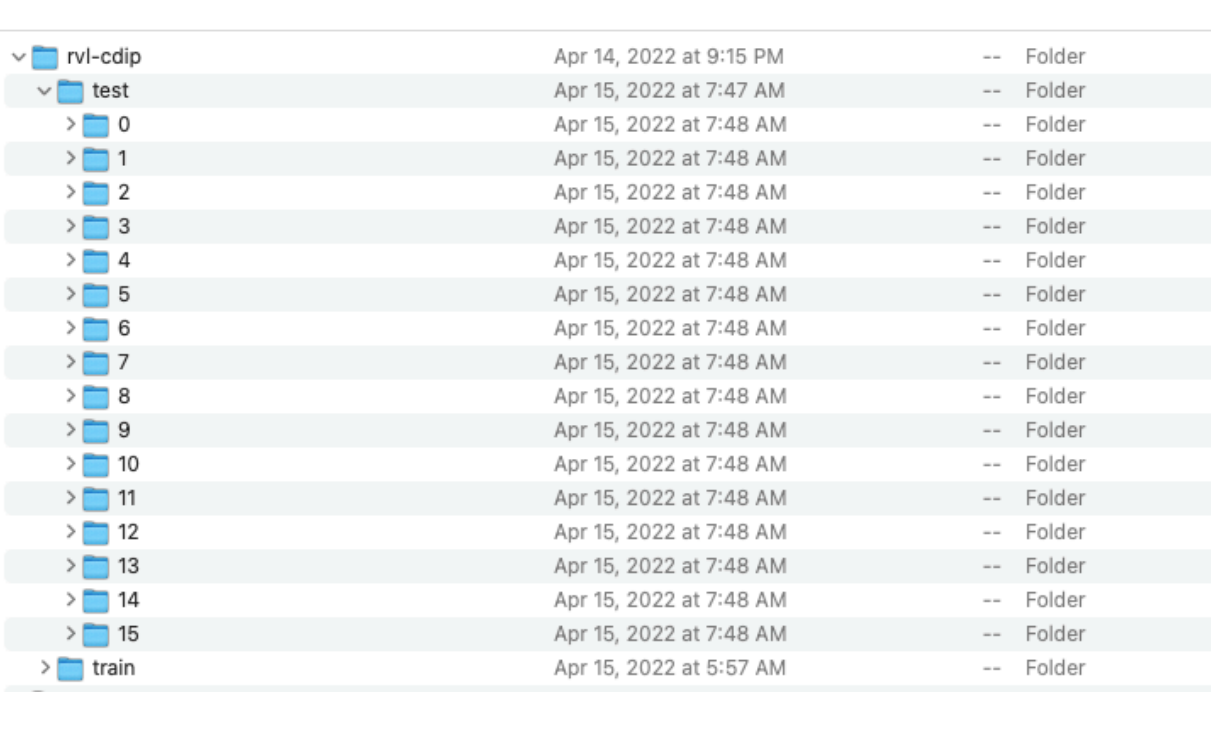
Uncompressed zip file contains directory structure as shown below. Labels directory contains files for training, test and validation data. The file has an image name, along with an associated category.



Name	Date Modified	Size
rvl-cdip	Yesterday at 1:25 PM	--
readme.txt	Jun 5, 2015 at 4:55 PM	2 KB
labels	Yesterday at 2:11 PM	--
val.txt	Dec 31, 2014 at 3:37 PM	1.7 MB
train.txt	Dec 31, 2014 at 3:37 PM	13.7 MB
test.txt	Dec 31, 2014 at 3:37 PM	1.7 MB
images	Yesterday at 1:25 PM	--

Note: The directory structure is complex in terms of accessing data to train, validate model. We need preprocessing of data before feed into model(s).

Preprocessing

> **Step 1. Reorganize images** There are 320,000 training images, 40,000 validation images, and 40,000 test images. Images need to reorganize such that it is easy to load using keras ImageDataGenerator. In this stpe, I will sort images into specific folder structure. The given dataset has nested/hierachical structure which is not readable or easily feed into data generator. The image generator requires directory structure as train/test with images under individual categoryies, as shown below:
>
> 
>
> | | | | |
> |---|---|---|---|
> | rvl-cdip | Apr 14, 2022 at 9:15 PM | -- | Folder |
> | test | Apr 15, 2022 at 7:47 AM | -- | Folder |
> | 0 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 1 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 2 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 3 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 4 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 5 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 6 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 7 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 8 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 9 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 10 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 11 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 12 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 13 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 14 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | 15 | Apr 15, 2022 at 7:48 AM | -- | Folder |
> | train | Apr 15, 2022 at 5:57 AM | -- | Folder |
>
> ***Note: I will use train and test images in this project.***

In [6]:

```python
# Imports
import warnings
import os, random
import shutil
import matplotlib.pyplot as plt
import numpy as np
import skimage.io as skio
import skimage.transform as sktran
import keras.models as models
import keras.layers as layers
from keras import regularizers
from keras.layers import Dropout
from keras import optimizers
import keras.preprocessing.image as kimage
import keras
```

In [ ]:

```python
# Sort images into train/test/validation directory
# Each dataset will have 0 to 15 directory, representing category.
# Each directory for a category will have all images for that category.

base_dir = "/Volumes/T7/rvl-cdip-orig/"
images_base = base_dir + "images/"

target_path = "/Volumes/T7/rvl-cdip/"

def _label_file(label_file, file_type):

    with open(label_file) as f:
        labels = f.readlines()
        print(f"Total labels {len(labels)}")
        i = 0
        for label in labels:
            image_path = label.split(' ')[0]
            image_name = image_path.split('/')[-1].rstrip('\n')
            category = label.split(' ')[1].rstrip()

            destination_path = target_path + file_type + category
            if not os.path.exists(destination_path):
                os.makedirs(destination_path)

            file_destination =  destination_path + "/" + image_name

            shutil.move(images_base+image_path,file_destination)
            i = i + 1
        print(f"Total files transfer {i}")
```

In [ ]:

```python
# Train images
train_label_file = base_dir + "labels/train.txt"
_label_file(train_label_file, "train/")
```

```
Total labels 320000
Total files transfer 320000
```

In [ ]:

```python
# Test images
train_label_file = base_dir + "labels/test.txt"
_label_file(train_label_file, "test/")
```

```
Total labels 40000
Total files transfer 40000
```

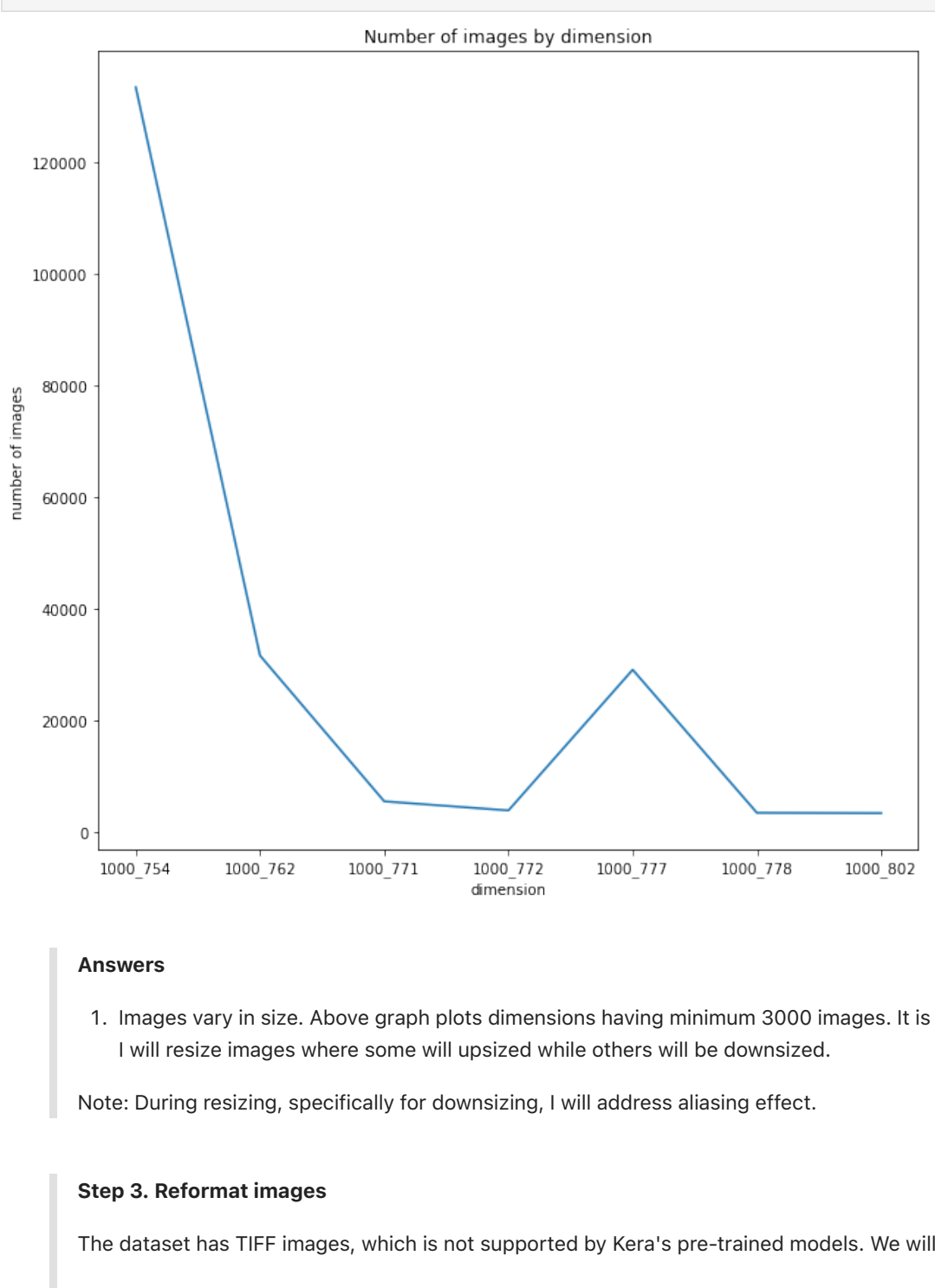
> **Step 2. Resize images**
>
> At this point, we have large dataset. We need to answer following question about given images:
>
> 1. Do they have different dimensions? If so, is there any distribution of images by dimension?

In [ ]:

```python
%%capture
# Find min and max dimensions across all images (train/test/validation)
base_path = "/Volumes/T7/rvl-cdip"

size_dir = {}

# Following loop iterates over each directories/subdirectories to create a dictionary of size wise occurances
for root, dirs, files in os.walk(base_path):
    for file in files:
        file_path = os.path.join(root, file)
        if '.tif' in file_path and "_" not in file_path:
            img = skio.imread(file_path)
            if img.shape in size_dir:
                size_dir[img.shape] = size_dir[img.shape] + 1
            else:
                size_dir[img.shape] = 1
```

In [ ]:

```python
# Show histogram to understand mostly available size
# This shows dimension with more than 3000 images
filtered_data = {}
for key, value in size_dir.items():
    if len(key) == 2 and value > 3000:
        filtered_data[str(key[0]) + "_" + str(key[1])] = value

lists = sorted(filtered_data.items())
x, y = zip(*lists)

plt.figure(figsize=(10,10))
plt.plot(x, y)
plt.xlabel("dimension")
plt.ylabel("number of images")
plt.title("Number of images by dimension")
plt.show()
```



> **Answers**
>
> 1. Images vary in size. Above graph plots dimensions having minimum 3000 images. It is clear that majority of images are around 1000 x 754. This single dimension (1000x754) has largest number of images, but it represents just about 46% of total images. I will resize images where some will upsized while others will be downsized.
>
> Note: During resizing, specifically for downsizing, I will address aliasing effect.

> **Step 3. Reformat images**
>
> The dataset has TIFF images, which is not supported by Kera's pre-trained models. We will convert images to PNG which will have two benefits.
>
> 1. Size of PNG will be much smaller than TIFF. This will help while uploading to Google Drive due to smaller size.
> 2. PNG will be used by Kera's pre-trained models.
>
> **Note:** Instead of doing one time exerciese of resize images, I have created a reusable function. This allows to recreate different sized dataset as we proceed with the project. Given computation and storage limitation, we may not able to process complete dataset.
>
> **Following function uses random sequence to select file from the directory. This is to overcome bias associated with ordering of file in the directory.**

In [ ]:

```python
%%capture
# Function to generate desired dataset
def generate_dataset(width, height, target_path, source_path, desired_size_per_category):
    category = 0
    for root, dirs, files in os.walk(source_path):

        if len(files) == 0:
            continue

        file_count = 0
        for file in files:
            file_path = os.path.join(root, file)
            if '.tif' in file_path and not "._" in file_path:
                file_count = file_count + 1

        i = 0
        print(f"Directory {root} has {file_count} files, we have to choose {desired_size_per_category} files (randomly).")

        # Add randomness in data generation.
        # Generate a random sequence. We will pickup files randomly
        file_seq = np.sort(random.sample(range(1, file_count), desired_size_per_category))

        index  = 1
        for file in files:
            file_path = os.path.join(root, file)

            if '.tif' in file_path and not "._" in file_path:
                index = index + 1
                if index in file_seq:
                    img = skio.imread(file_path)
                    if img.size != 0:
                        resized_image = sktran.resize(img, (width, height), anti_aliasing=True)

                        target_file = file_path.replace("/Volumes/T7/rvl-cdip", target_path)
                        destination_dir = os.path.split(target_file)[0]
                        if not os.path.exists(destination_dir):
                            os.makedirs(destination_dir)

                        target_file = target_file.replace('.tif', '.png')
                        skio.imsave(target_file, (resized_image * 255).astype(np.uint8), check_contrast=False)
                        i = i + 1
                        if i > desired_size_per_category:
                            print(f"Category {category} has {i} images ")
                            category = category + 1
                            break
```

> **Summary:** Above function allows to generate a dataset for given height, width, and number of images per categories. It will generate the dataset in the structure which can be directly feed into ImageDataGenerator. Also, it converts TIF file to PNG, which helps while using pretrained models.
>
> Note: The function generates dataset by selecting images randomly from available images for the category.

Infrastructure details

- we are going to use Colab Pro+ for this project. GPU and memory details are as below.
- Google drive storage is 200GB subscription.

In [ ]:

```python
gpu_info = !nvidia-smi
gpu_info = '\n'.join(gpu_info)
if gpu_info.find('failed') >= 0:
  print('Not connected to a GPU')
else:
  print(gpu_info)


from psutil import virtual_memory
ram_gb = virtual_memory().total / 1e9
print('Your runtime has {:.1f} gigabytes of available RAM\n'.format(ram_gb))

if ram_gb < 20:
  print('Not using a high-RAM runtime')
else:
  print('You are using a high-RAM runtime!')
```

```
Mon May  2 11:02:42 2022
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 460.32.03    Driver Version: 460.32.03    CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla P100-PCIE...  Off  | 00000000:00:04.0 Off |                    0 |
| N/A   41C    P0    28W / 250W |      0MiB / 16280MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
Your runtime has 54.8 gigabytes of available RAM

You are using a high-RAM runtime!
```

Dataset size

>

I have taken an iterative approach, where different datasets (100, 500, 1000, 2000, and 10000 images per category) are generated. To avoid repeating sections in this notebook, I have included only three datasets for demonstration purpose.

Dataset: 16,000 This dataset includes 1000 images per categoy for training the model and 200 images per cateogr for validation. All images are 1000x768 size

In [ ]:

```python
# Generate train dataset for 1000x768 with 1000 images for each category
source_path = "/Volumes/T7/rvl-cdip/train"
target_path = "/Volumes/T7/sample_1000/rvl-cdip/"
desired_size_per_category = 1000
width = 1000
height = 768

generate_dataset(width, height, target_path, source_path, desired_size_per_category)
```

```
Directory /Volumes/T7/rvl-cdip/train/15 has 19975 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/1 has 19957 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/7 has 19997 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/10 has 20010 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/3 has 20031 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/14 has 20006 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/11 has 19944 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/13 has 20042 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/0 has 20103 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/2 has 19954 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/9 has 19987 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/4 has 19963 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/12 has 20043 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/5 has 19984 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/6 has 19829 files, we have to choose 1000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/8 has 20012 files, we have to choose 1000 files (randomly).
```

Test images (3200) 200/category

In [ ]:

```python
# Generate test dataset for 1000x768 with 200 images for each category

source_path = "/Volumes/T7/rvl-cdip/test"
target_path = "/Volumes/T7/sample_1000/rvl-cdip/"
desired_size_per_category = 200
width = 1000
height = 768

generate_dataset(width, height, target_path, source_path, desired_size_per_category)
```

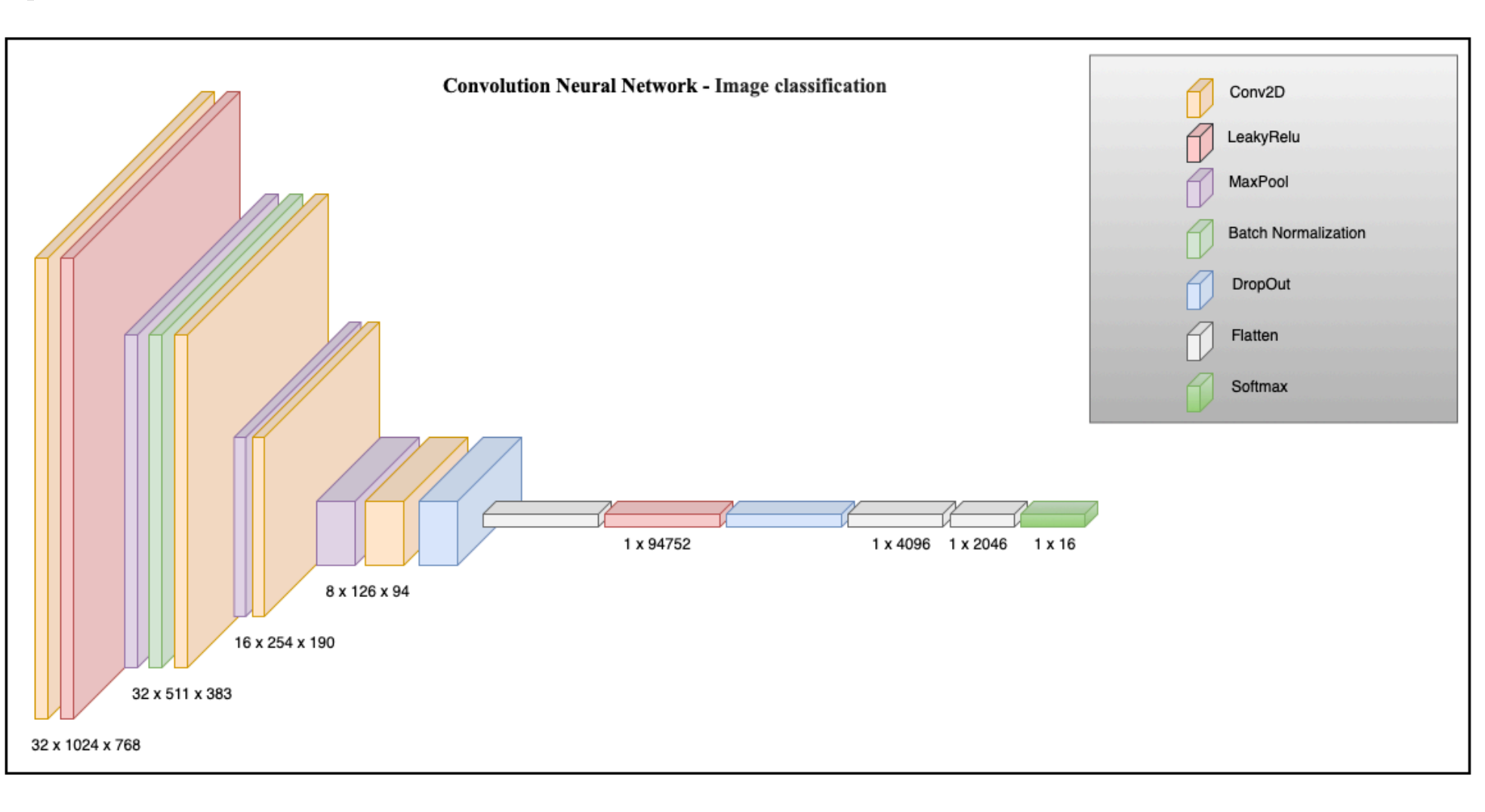
```
Directory /Volumes/T7/rvl-cdip/test/3 has 2532 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/14 has 2536 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/2 has 2516 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/9 has 2463 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/6 has 2570 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/13 has 2435 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/8 has 2527 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/0 has 2464 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/7 has 2472 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/5 has 2498 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/1 has 2506 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/15 has 2492 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/11 has 2477 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/10 has 2505 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/12 has 2489 files, we have to choose 200 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/4 has 2515 files, we have to choose 200 files (randomly).
```

Modeling

> I will build a model using Convolution Neural Network(CNN). I will also create two other models using transfer learning from EfficientNetB0 as well as Resnet50.
> I will generate different datasets for different sizes. I will feed them to same three models as genearted below. For every dataset, I will collect metrics for all three models.
>
> **Objective**: This will allow to choose the model based on metrics collected for different dataset size.
>
> **Approach at high level:**
>
> 1. Model with small dataset: (100 images/category) Model name: 1,600
>   A. Basic model with few Convolution layers using RELU, and softmax activation functions.
>   B. EfficientNetB0 for same dataset to evaluate and capture metrics.
>   C. ResNet50 for same dataset to evaluate and capture metrics.
> 2. Model with small dataset: (500 images/category) Model name: 8,000
>   A. Evaluate and capture metrics for CNN model generated at step 1.1.
>   B. Evaluate and capture metrics for EfficientNetB0 generated at step 1.2.
>   C. Evaluate and capture metrics for ResNet50 generated at step 1.3.
> 3. Model with moderate dataset: (1000 images/category) Model name: 16,000
>   A. Evaluate and capture metrics for CNN model generated at step 1.1.
>   B. Evaluate and capture metrics for EfficientNetB0 generated at step 1.2.
>   C. Evaluate and capture metrics for ResNet50 generated at step 1.3.
> 4. Model with moderate dataset: (2000 images/category) Model name: 32,000
>   A. Evaluate and capture metrics for CNN model generated at step 1.1.
>   B. Evaluate and capture metrics for EfficientNetB0 generated at step 1.2.
>   C. Evaluate and capture metrics for ResNet50 generated at step 1.3.
> 5. Model with large dataset: (10000 images/category) Model name: 160,000
>   A. Evaluate and capture metrics for CNN model generated at step 1.1.
>   B. Evaluate and capture metrics for EfficientNetB0 generated at step 1.2.
>   C. Evaluate and capture metrics for ResNet50 generated at step 1.3.
>
> **All models will have early stopping callback on validation accuracy with patience of 4 epochs. If model does not make any progress after 4 epochs on validation accuracy, it will stop.**

Convolution Neural Network

> Building CNN to classify document image for given dataset.



> **High level summary:** As shown above, I am strating with input image with 32 channels. I am using Rectified Linear Unit (ReLU) activation function. I am adding Max-Pool layer to select the maximum element from the region of the feature map covered by the kernel. The output of max pooling will have reduced dimensionality (half of input). I am using L2 regularization along with Dropout layers to avoid overfitting of the model. The output of the model is based on softmax.

In [5]:

```python
nn = models.Sequential()

#  3X3 convolutional layer with 32 output channels using the Conv2D function, with activation='relu'
nn.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(1024, 768, 1)))

nn.add(layers.LeakyReLU(alpha=0.2))

# A 3X3 max pooling layer using the MaxPooling2D function.
nn.add(layers.MaxPooling2D((2, 2)))

nn.add(layers.BatchNormalization(momentum=0.8))

# A final 3X3 convolution layer with 64 output channels, with activation='relu'.
nn.add(layers.Conv2D(16, (3, 3), activation='relu'))

# Another 2X2 max pooling layer.
nn.add(layers.MaxPooling2D((2, 2)))

# A final 3X3 convolution layer with 16 output channels, with activation='relu'.
nn.add(layers.Conv2D(8, (3, 3), activation='relu'))

nn.add(layers.MaxPooling2D((2, 2)))

nn.add(layers.Dropout(rate=0.10))

# The 64 channel feature map is flattened to a vector.
nn.add(layers.Flatten())

nn.add(layers.LeakyReLU(alpha=0.1))

# A Dense (fully-connected) hidden layer with 64 units is the first classifier layer, with activation='relu' , and L1 regularization
nn.add(layers.Dense(units=16, activation='relu', kernel_regularizer=regularizers.L2(l2=0.001)))

# A dropout layer is used to improve regularization of the model using the Dropout function with argument 0.5.
nn.add(layers.Dropout(rate=0.15))

nn.add(layers.Dense(units=4096, activation='relu'))

nn.add(layers.Dense(units=2048, activation='relu'))

# The output Dense layer has 10 units with activation='softmax' to indicate the classification of the digits.
nn.add(layers.Dense(units=16, activation='softmax'))

nn.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 1022, 766, 32)     320

 leaky_re_lu (LeakyReLU)     (None, 1022, 766, 32)     0

 max_pooling2d (MaxPooling2D  (None, 511, 383, 32)     0
 )

 batch_normalization (BatchN  (None, 511, 383, 32)     128
 ormalization)

 conv2d_1 (Conv2D)           (None, 509, 381, 16)      4624

 max_pooling2d_1 (MaxPooling  (None, 254, 190, 16)     0
 2D)

 conv2d_2 (Conv2D)           (None, 252, 188, 8)       1160

 max_pooling2d_2 (MaxPooling  (None, 126, 94, 8)       0
 2D)

 dropout (Dropout)           (None, 126, 94, 8)        0

 flatten (Flatten)           (None, 94752)             0

 leaky_re_lu_1 (LeakyReLU)   (None, 94752)             0

 dense (Dense)               (None, 16)                1516048

 dropout_1 (Dropout)         (None, 16)                0

 dense_1 (Dense)             (None, 4096)              69632

 dense_2 (Dense)             (None, 2048)              8390656

 dense_3 (Dense)             (None, 16)                32784

=================================================================
Total params: 10,015,352
Trainable params: 10,015,288
Non-trainable params: 64
_________________________________________________________________
```

In [6]:

```python
## Compile the model
nn.compile(optimizer = 'RMSprop', loss = 'categorical_crossentropy',
                metrics = ['accuracy'])
```

Early stopping condition on Validation Accuracy

In [11]:

```python
# Early stopping if no improvement noticed
callbacks_list = [
        keras.callbacks.EarlyStopping(
            monitor = 'val_accuracy', # Use accuracy to monitor the model
            patience = 4 # Stop after one step with lower accuracy
        )
]
```

In [ ]:

```python
datagen = kimage.ImageDataGenerator()

# load and iterate training dataset
train_it = datagen.flow_from_directory('/content/sample_1000/rvl-cdip/train/', class_mode='categorical', target_size=(1024, 768), color_mode='grayscale')
# load and iterate test datase
test_it = datagen.flow_from_directory('/content/sample_1000/rvl-cdip/test/', class_mode='categorical',  target_size=(1024, 768), color_mode='grayscale')
```

```
Found 15999 images belonging to 16 classes.
Found 3199 images belonging to 16 classes.
```

>

> ## fit model with dataset size: 16,000 train and 3,200 validation images

In [ ]:

```python
## Now fit the model
simple_overfitting_model = nn.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

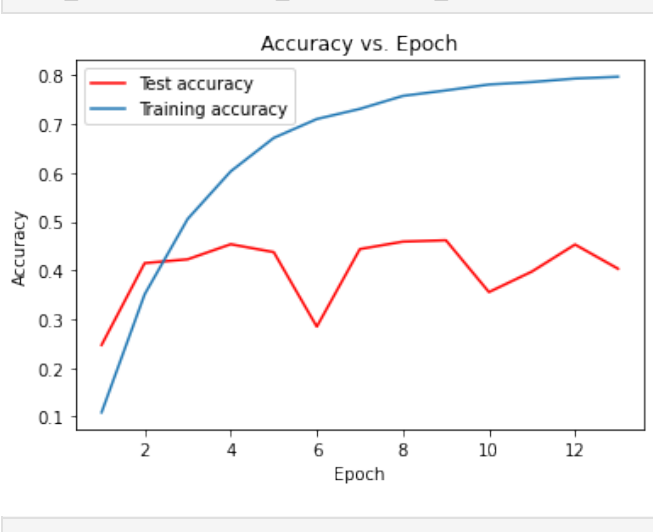
```
Epoch 1/40
500/500 [==============================] - 192s 361ms/step - loss: 3.3842 - accuracy: 0.1087 - val_loss: 2.3674 - val_accuracy: 0.2470
Epoch 2/40
500/500 [==============================] - 175s 349ms/step - loss: 2.1399 - accuracy: 0.3508 - val_loss: 2.0427 - val_accuracy: 0.4148
Epoch 3/40
500/500 [==============================] - 170s 339ms/step - loss: 1.7714 - accuracy: 0.5055 - val_loss: 2.0621 - val_accuracy: 0.4226
Epoch 4/40
500/500 [==============================] - 169s 338ms/step - loss: 1.5303 - accuracy: 0.6030 - val_loss: 2.0255 - val_accuracy: 0.4536
Epoch 5/40
500/500 [==============================] - 169s 338ms/step - loss: 1.3564 - accuracy: 0.6717 - val_loss: 2.1388 - val_accuracy: 0.4373
Epoch 6/40
500/500 [==============================] - 170s 339ms/step - loss: 1.2703 - accuracy: 0.7104 - val_loss: 2.8509 - val_accuracy: 0.2845
Epoch 7/40
500/500 [==============================] - 170s 339ms/step - loss: 1.2178 - accuracy: 0.7312 - val_loss: 2.2169 - val_accuracy: 0.4436
Epoch 8/40
500/500 [==============================] - 169s 337ms/step - loss: 1.1305 - accuracy: 0.7578 - val_loss: 2.1726 - val_accuracy: 0.4589
Epoch 9/40
500/500 [==============================] - 169s 337ms/step - loss: 1.0826 - accuracy: 0.7690 - val_loss: 2.3664 - val_accuracy: 0.4617
Epoch 10/40
500/500 [==============================] - 169s 337ms/step - loss: 1.0510 - accuracy: 0.7812 - val_loss: 2.5445 - val_accuracy: 0.3554
Epoch 11/40
500/500 [==============================] - 169s 337ms/step - loss: 1.0261 - accuracy: 0.7863 - val_loss: 2.4970 - val_accuracy: 0.3976
Epoch 12/40
500/500 [==============================] - 170s 340ms/step - loss: 0.9980 - accuracy: 0.7934 - val_loss: 2.2735 - val_accuracy: 0.4530
Epoch 13/40
500/500 [==============================] - 170s 339ms/step - loss: 0.9889 - accuracy: 0.7971 - val_loss: 2.4501 - val_accuracy: 0.4033
```

In [10]:

```python
def plot_loss(history):
    train_loss = history.history['loss']
    test_loss = history.history['val_loss']
    x = list(range(1, len(test_loss) + 1))
    plt.plot(x, test_loss, color='red', label='Test loss')
    plt.plot(x, train_loss, color='blue', label='Training loss')
    plt.xlabel('Epoch')
    plt.ylabel('Training Loss')
    plt.title('Loss vs. Epoch')
    plt.legend()
    plt.show()

def plot_accuracy(history):
    train_acc = history.history['accuracy']
    test_acc = history.history['val_accuracy']
    x = list(range(1, len(test_acc) + 1))
    plt.plot(x, test_acc, color = 'red', label = 'Test accuracy')
    plt.plot(x, train_acc, label = 'Training accuracy')
    plt.legend()
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Accuracy vs. Epoch')
```

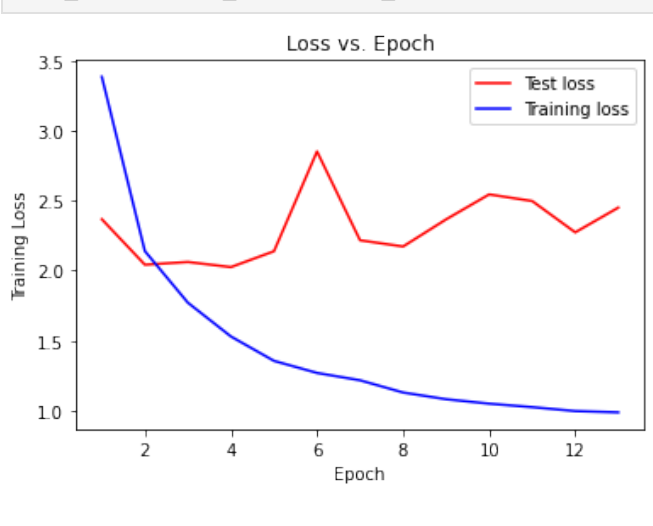
In [ ]:

```python
plot_accuracy(simple_overfitting_model)
```



In [ ]:

```python
plot_loss(simple_overfitting_model)
```



Summary: (CNN)

> Model metrics: Model name: 16,000

> > Highest training accuracy : 0.7971
> >
> > Lowest training loss: 0.9889
> >
> > Highest validation accuracy: 0.4617
> >
> > Lowest validation loss: 2.0255

EfficientNetB0

> I have tried different EfficientNet (B0, B1 and B6) models. But due to computation and memory limit, I am using EfficientNetB0 in this project.

In [7]:
```
# Input models and associated packages
import keras.utils.np_utils as ku
import keras.models as models
import keras.layers as layers
from keras import regularizers
from keras.layers import Dropout
from keras import optimizers
```

In [8]:
```
# Import pre-trained models
from keras.layers import Conv2D, Input
from tensorflow.keras.applications import *
```

In [16]:
```
# EfficientNet model
conv_net= EfficientNetB0(weights='imagenet', include_top=False, input_shape=(500, 384, 3))
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16711680/16705208 [==============================] - 0s 0us/step
16719872/16705208 [==============================] - 0s 0us/step
```

In [19]:
```
# Pre-trained model with additional convolution layers.

small_efficientNetB0_model = models.Sequential()
small_efficientNetB0_model.add(layers.Conv2D(3, (3, 3), activation='relu', input_shape=(1000, 768, 1), padding="same"))
small_efficientNetB0_model.add(layers.MaxPooling2D((2, 2)))
small_efficientNetB0_model.add(layers.LeakyReLU(alpha=0.25))
small_efficientNetB0_model.add(conv_net)
small_efficientNetB0_model.add(layers.GlobalAveragePooling2D())
small_efficientNetB0_model.add(layers.Dense(units=2048, activation='relu'))
small_efficientNetB0_model.add(layers.Dense(16, activation="softmax"))
# Making trainable false for pre-trained model
conv_net.trainable=False
small_efficientNetB0_model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #   
=================================================================
 conv2d_3 (Conv2D)           (None, 1000, 768, 3)      30        
                                                                 
 max_pooling2d_3 (MaxPooling  (None, 500, 384, 3)      0         
 2D)                                                             
                                                                 
 leaky_re_lu_2 (LeakyReLU)   (None, 500, 384, 3)       0         
                                                                 
 efficientnetb0 (Functional)  (None, 16, 12, 1280)     4049571   
                                                                 
 global_average_pooling2d (G  (None, 1280)             0         
 lobalAveragePooling2D)                                          
                                                                 
 dense_4 (Dense)             (None, 2048)              2623488   
                                                                 
 dense_5 (Dense)             (None, 16)                32784     
                                                                 
=================================================================
Total params: 6,705,873
Trainable params: 2,656,302
Non-trainable params: 4,049,571
_________________________________________________________________
```

In [20]:
```
small_efficientNetB0_model.compile(optimizer = 'RMSprop', loss = 'categorical_crossentropy', metrics = ['accuracy'])
```

In [12]:
```
# Connect to google drive
from google.colab import drive
drive.mount("/content/drive", force_remount=True)
```

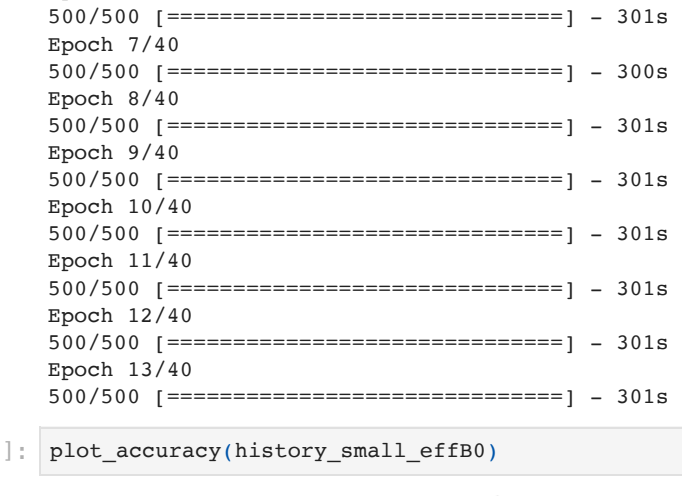
```
Mounted at /content/drive
```

In [ ]:
```
datagen = kimage.ImageDataGenerator()

# load and iterate training dataset
train_it = datagen.flow_from_directory('/content/sample_1000/rvl-cdip/train/', class_mode='categorical', target_size=(1024, 768), color_mode='grayscale')
# load and iterate test datase
test_it = datagen.flow_from_directory('/content/sample_1000/rvl-cdip/test/', class_mode='categorical',  target_size=(1024, 768), color_mode='grayscale')
```

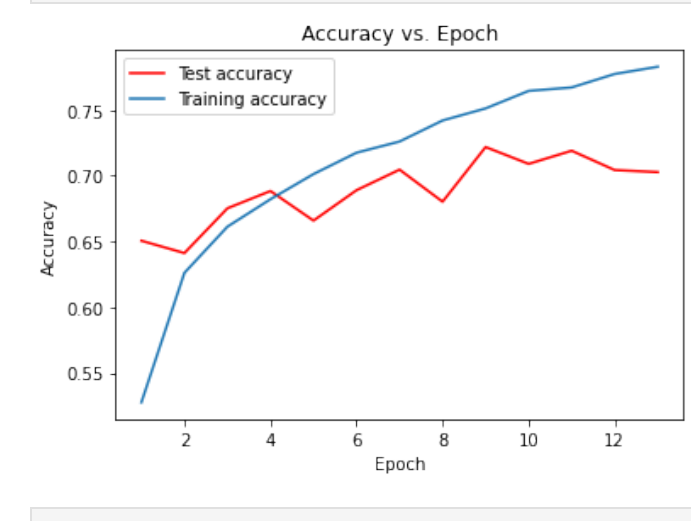
In [ ]:
```
history_small_effB0 = small_efficientNetB0_model.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
500/500 [==============================] - 314s 608ms/step - loss: 1.5370 - accuracy: 0.5276 - val_loss: 1.1832 - val_accuracy: 0.6505
Epoch 2/40
500/500 [==============================] - 302s 604ms/step - loss: 1.2124 - accuracy: 0.6261 - val_loss: 1.1806 - val_accuracy: 0.6411
Epoch 3/40
500/500 [==============================] - 301s 601ms/step - loss: 1.1006 - accuracy: 0.6612 - val_loss: 1.0742 - val_accuracy: 0.6752
Epoch 4/40
500/500 [==============================] - 301s 601ms/step - loss: 1.0270 - accuracy: 0.6821 - val_loss: 1.0882 - val_accuracy: 0.6883
Epoch 5/40
500/500 [==============================] - 300s 600ms/step - loss: 0.9664 - accuracy: 0.7012 - val_loss: 1.1239 - val_accuracy: 0.6658
Epoch 6/40
500/500 [==============================] - 301s 601ms/step - loss: 0.9149 - accuracy: 0.7175 - val_loss: 1.0921 - val_accuracy: 0.6890
Epoch 7/40
500/500 [==============================] - 300s 600ms/step - loss: 0.8797 - accuracy: 0.7260 - val_loss: 1.0263 - val_accuracy: 0.7046
Epoch 8/40
500/500 [==============================] - 301s 601ms/step - loss: 0.8357 - accuracy: 0.7420 - val_loss: 1.1810 - val_accuracy: 0.6802
Epoch 9/40
500/500 [==============================] - 301s 601ms/step - loss: 0.7971 - accuracy: 0.7512 - val_loss: 1.0092 - val_accuracy: 0.7218
Epoch 10/40
500/500 [==============================] - 301s 602ms/step - loss: 0.7706 - accuracy: 0.7645 - val_loss: 1.0968 - val_accuracy: 0.7090
Epoch 11/40
500/500 [==============================] - 301s 602ms/step - loss: 0.7443 - accuracy: 0.7671 - val_loss: 1.1366 - val_accuracy: 0.7190
Epoch 12/40
500/500 [==============================] - 301s 602ms/step - loss: 0.7139 - accuracy: 0.7774 - val_loss: 1.1972 - val_accuracy: 0.7043
Epoch 13/40
500/500 [==============================] - 301s 602ms/step - loss: 0.6986 - accuracy: 0.7829 - val_loss: 1.2036 - val_accuracy: 0.7027
```

In [ ]:
```
plot_accuracy(history_small_effB0)
```



In [ ]:
```
plot_loss(history_small_effB0)
```



Summary: (EfficientNetB0)

> Model metrics: Model name: 16,000
>
> > Highest training accuracy : 0.7829
> >
> > Lowest training loss: 0.6986
> >
> > Highest validation accuracy: 0.7218
> >
> > Lowest validation loss: 1.0092

ResNet50

> We will try same with ResNet50 to evaluate if it provides accuracy better than EfficientNetB0. I have tried ResNet50, ResNet101 and ResNet152. For small dataset, models with ResNet101 and ResNet152 run on Colab Pro+ with GPU option. But, colab pro throws Out Of Memory error as dataset size grows 1000 or more images per category. Model based on ResNet50 runs for all datasets so, I have used it for this project.

In [9]:
```
# ResNet101 model
res_net= ResNet50(weights='imagenet', include_top=False, input_shape=(500, 384, 3))
```

```
2022-05-07 09:14:46.535716: I tensorflow/core/platform/cpu_feature_guard.cc:151] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critica
l operations:  AVX2 FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94773248/94765736 [==============================] - 2s 0us/step
94781440/94765736 [==============================] - 2s 0us/step
```

In [10]:
```
# Pre-trained model with additional convolution layers.

small_resnet50_model = models.Sequential()
small_resnet50_model.add(layers.Conv2D(3, (3, 3), activation='relu', input_shape=(1000, 768, 1), padding="same"))
small_resnet50_model.add(layers.MaxPooling2D((2, 2)))
small_resnet50_model.add(layers.LeakyReLU(alpha=0.25))
small_resnet50_model.add(res_net)
small_resnet50_model.add(layers.GlobalAveragePooling2D())
small_resnet50_model.add(layers.Dense(units=2048, activation='relu'))
small_resnet50_model.add(layers.Dense(16, activation="softmax"))
# Making trainable false for pre-trained model
res_net.trainable=False
small_resnet50_model.summary()
```

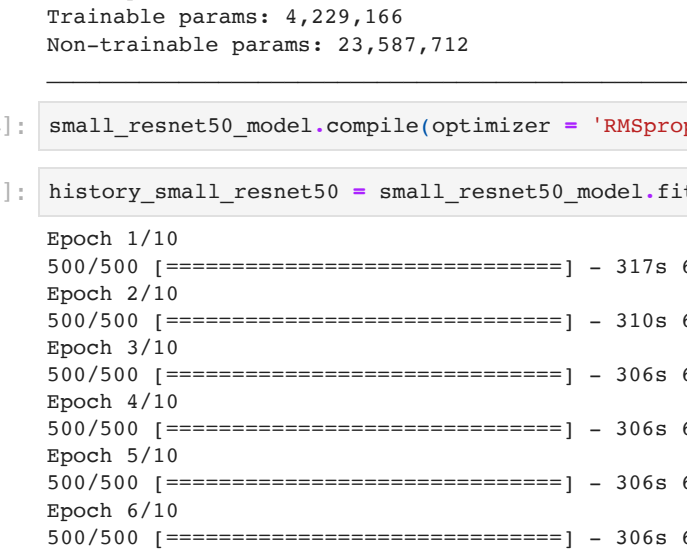
```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #   
=================================================================
 conv2d (Conv2D)             (None, 1000, 768, 3)      30        
                                                                 
 max_pooling2d (MaxPooling2D  (None, 500, 384, 3)      0         
 )                                                               
                                                                 
 leaky_re_lu (LeakyReLU)     (None, 500, 384, 3)       0         
                                                                 
 resnet50 (Functional)       (None, 16, 12, 2048)      23587712  
                                                                 
 global_average_pooling2d (G  (None, 2048)             0         
 lobalAveragePooling2D)                                          
                                                                 
 dense (Dense)               (None, 2048)              4196352   
                                                                 
 dense_1 (Dense)             (None, 16)                32784     
                                                                 
=================================================================
Total params: 27,816,878
Trainable params: 4,229,166
Non-trainable params: 23,587,712
_________________________________________________________________
```

In [14]:
```
small_resnet50_model.compile(optimizer = 'RMSprop', loss = 'categorical_crossentropy', metrics = ['accuracy'])
```

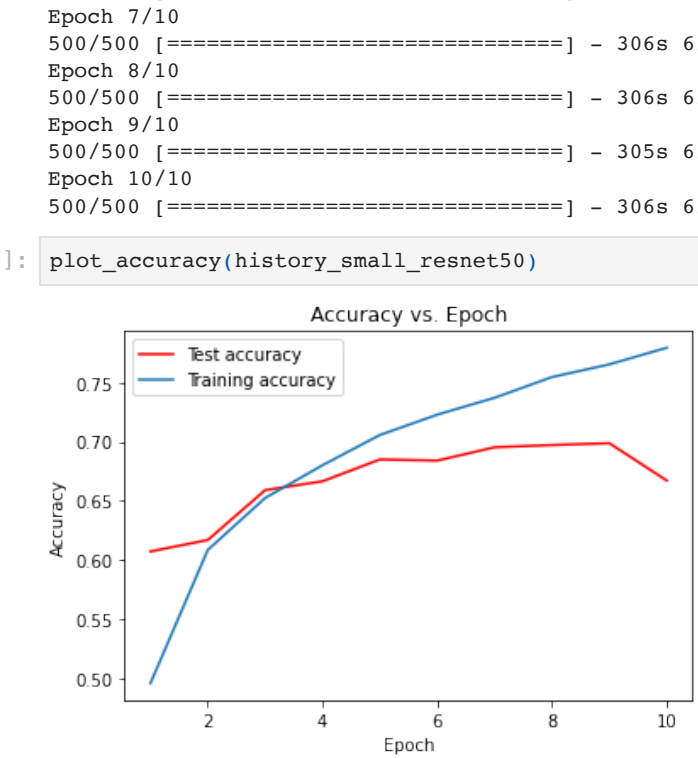
In [ ]:
```
history_small_resnet50 = small_resnet50_model.fit(train_it, epochs=10, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/10
500/500 [==============================] - 317s 623ms/step - loss: 1.7876 - accuracy: 0.4955 - val_loss: 1.2868 - val_accuracy: 0.6071
Epoch 2/10
500/500 [==============================] - 310s 619ms/step - loss: 1.2569 - accuracy: 0.6084 - val_loss: 1.2210 - val_accuracy: 0.6168
Epoch 3/10
500/500 [==============================] - 306s 611ms/step - loss: 1.1253 - accuracy: 0.6523 - val_loss: 1.1180 - val_accuracy: 0.6590
Epoch 4/10
500/500 [==============================] - 306s 611ms/step - loss: 1.0354 - accuracy: 0.6800 - val_loss: 1.1728 - val_accuracy: 0.6665
Epoch 5/10
500/500 [==============================] - 306s 611ms/step - loss: 0.9542 - accuracy: 0.7056 - val_loss: 1.0824 - val_accuracy: 0.6849
Epoch 6/10
500/500 [==============================] - 306s 611ms/step - loss: 0.9027 - accuracy: 0.7228 - val_loss: 1.1104 - val_accuracy: 0.6840
Epoch 7/10
500/500 [==============================] - 306s 611ms/step - loss: 0.8466 - accuracy: 0.7371 - val_loss: 1.0930 - val_accuracy: 0.6952
Epoch 8/10
500/500 [==============================] - 306s 611ms/step - loss: 0.8029 - accuracy: 0.7546 - val_loss: 1.0622 - val_accuracy: 0.6971
Epoch 9/10
500/500 [==============================] - 305s 610ms/step - loss: 0.7618 - accuracy: 0.7654 - val_loss: 1.1264 - val_accuracy: 0.6987
Epoch 10/10
500/500 [==============================] - 306s 611ms/step - loss: 0.7077 - accuracy: 0.7794 - val_loss: 1.3106 - val_accuracy: 0.6671
```

In [ ]:
```
plot_accuracy(history_small_resnet50)
```



In [ ]:
```
plot_loss(history_small_resnet50)
```



Summary: (ResNet50)

> Model metrics: Model name: 16,000
>
> > Highest training accuracy : 0.7794
> >
> > Lowest training loss: 0.7618
> >
> > Highest validation accuracy: 0.6987
> >
> > Lowest validation loss: 1.0824

>

fit model with dataset size: 32,000 train and 6,400 validation images

In [ ]:
```
# Generate train dataset for 1000x768 with 2000 images for each category
source_path = "/Volumes/T7/rvl-cdip/train"
target_path = "/Volumes/T7/sample_2000/rvl-cdip/"
desired_size_per_category = 2000
width = 1000
height = 768

generate_dataset(width, height, target_path, source_path, desired_size_per_category)
```

```
Directory /Volumes/T7/rvl-cdip/train/15 has 19975 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/1 has 19957 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/7 has 19997 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/10 has 20010 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/3 has 20031 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/14 has 20006 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/11 has 19944 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/13 has 20042 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/0 has 20103 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/2 has 19954 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/9 has 19987 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/4 has 19963 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/12 has 20043 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/5 has 19984 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/6 has 19829 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/8 has 20012 files, we have to choose 2000 files (randomly).
```

In [ ]:
```
# Generate test dataset for 1000x768 with 1000 images for each category

source_path = "/Volumes/T7/rvl-cdip/test"
target_path = "/Volumes/T7/sample_5000/rvl-cdip/"
desired_size_per_category = 400
width = 1000
height = 768

generate_dataset(width, height, target_path, source_path, desired_size_per_category)
```

```
Directory /Volumes/T7/rvl-cdip/test/3 has 2532 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/14 has 2536 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/2 has 2516 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/9 has 2463 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/6 has 2570 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/13 has 2435 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/8 has 2527 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/0 has 2464 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/7 has 2472 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/5 has 2498 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/1 has 2506 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/15 has 2492 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/11 has 2477 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/10 has 2505 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/12 has 2489 files, we have to choose 400 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/4 has 2515 files, we have to choose 400 files (randomly).
```

In [ ]:
```
datagen = kimage.ImageDataGenerator()

# load and iterate training dataset
train_it = datagen.flow_from_directory('/content/sample_2000/rvl-cdip/train/', class_mode='categorical', target_size=(1024, 768), color_mode='grayscale')
# load and iterate test datase
test_it = datagen.flow_from_directory('/content/sample_2000/rvl-cdip/test/', class_mode='categorical',  target_size=(1024, 768), color_mode='grayscale')
```

```
Found 31997 images belonging to 16 classes.
Found 6398 images belonging to 16 classes.
```

In [13]:
```
# Early stopping if no improvement noticed
callbacks_list = [
        keras.callbacks.EarlyStopping(
            monitor = 'val_accuracy', # Use accuracy to monitor the model
            patience = 4 # Stop after one step with lower accuracy
        )
]
```

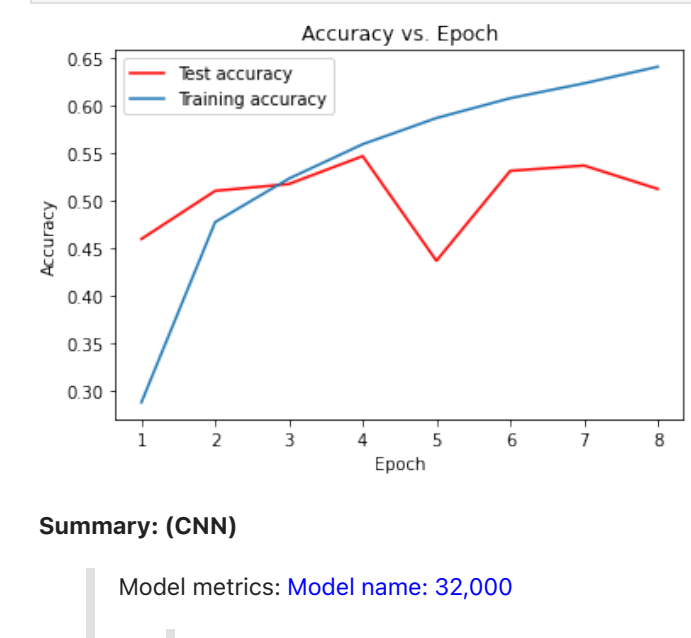
CNN

> Fit the model for given dataset (32,000)

In [ ]:
```
moderate_model = nn.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
1000/1000 [==============================] - 342s 338ms/step - loss: 2.5561 - accuracy: 0.2882 - val_loss: 1.8430 - val_accuracy: 0.4595
Epoch 2/40
1000/1000 [==============================] - 331s 330ms/step - loss: 1.8515 - accuracy: 0.4772 - val_loss: 1.7393 - val_accuracy: 0.5100
Epoch 3/40
1000/1000 [==============================] - 329s 329ms/step - loss: 1.7509 - accuracy: 0.5228 - val_loss: 1.8132 - val_accuracy: 0.5172
Epoch 4/40
1000/1000 [==============================] - 329s 329ms/step - loss: 1.6795 - accuracy: 0.5589 - val_loss: 1.7471 - val_accuracy: 0.5464
Epoch 5/40
1000/1000 [==============================] - 329s 329ms/step - loss: 1.6193 - accuracy: 0.5864 - val_loss: 2.0551 - val_accuracy: 0.4367
Epoch 6/40
1000/1000 [==============================] - 328s 328ms/step - loss: 1.5536 - accuracy: 0.6072 - val_loss: 1.7775 - val_accuracy: 0.5309
Epoch 7/40
1000/1000 [==============================] - 328s 328ms/step - loss: 1.5172 - accuracy: 0.6227 - val_loss: 1.7899 - val_accuracy: 0.5366
Epoch 8/40
1000/1000 [==============================] - 329s 328ms/step - loss: 1.4737 - accuracy: 0.6402 - val_loss: 1.8484 - val_accuracy: 0.5120
```

In [ ]:
```
plot_accuracy(moderate_model)
```



Summary: (CNN)

> Model metrics: Model name: 32,000
>
> > Highest training accuracy : 0.6402
> >
> > Lowest training loss: 1.4737
> >
> > Highest validation accuracy: 0.5464
> >
> > Lowest validation loss: 0.4367

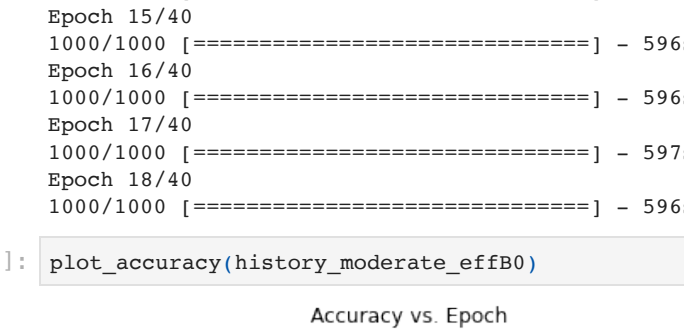
EfficientNetB0

> Fit the model for given dataset (32,000)

In [ ]:
```
history_moderate_effB0 = small_efficientNetB0_model.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
1000/1000 [==============================] - 610s 601ms/step - loss: 1.4148 - accuracy: 0.5599 - val_loss: 1.1669 - val_accuracy: 0.6407
Epoch 2/40
1000/1000 [==============================] - 597s 596ms/step - loss: 1.1393 - accuracy: 0.6513 - val_loss: 1.1196 - val_accuracy: 0.6533
Epoch 3/40
1000/1000 [==============================] - 597s 596ms/step - loss: 1.0412 - accuracy: 0.6838 - val_loss: 1.0597 - val_accuracy: 0.6879
Epoch 4/40
1000/1000 [==============================] - 597s 596ms/step - loss: 0.9784 - accuracy: 0.7013 - val_loss: 1.0512 - val_accuracy: 0.7049
Epoch 5/40
1000/1000 [==============================] - 597s 596ms/step - loss: 0.9272 - accuracy: 0.7196 - val_loss: 1.0915 - val_accuracy: 0.6927
Epoch 6/40
1000/1000 [==============================] - 597s 596ms/step - loss: 0.8924 - accuracy: 0.7294 - val_loss: 1.0150 - val_accuracy: 0.7260
Epoch 7/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.8644 - accuracy: 0.7411 - val_loss: 1.0500 - val_accuracy: 0.7165
Epoch 8/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.8302 - accuracy: 0.7494 - val_loss: 1.0678 - val_accuracy: 0.7108
Epoch 9/40
1000/1000 [==============================] - 597s 597ms/step - loss: 0.8048 - accuracy: 0.7559 - val_loss: 1.1139 - val_accuracy: 0.7219
Epoch 10/40
1000/1000 [==============================] - 597s 596ms/step - loss: 0.7807 - accuracy: 0.7650 - val_loss: 1.1027 - val_accuracy: 0.7313
Epoch 11/40
1000/1000 [==============================] - 596s 595ms/step - loss: 0.7611 - accuracy: 0.7665 - val_loss: 1.1764 - val_accuracy: 0.7123
Epoch 12/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.7345 - accuracy: 0.7781 - val_loss: 1.1308 - val_accuracy: 0.7371
Epoch 13/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.7200 - accuracy: 0.7817 - val_loss: 1.1621 - val_accuracy: 0.7334
Epoch 14/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.7022 - accuracy: 0.7894 - val_loss: 1.1664 - val_accuracy: 0.7509
Epoch 15/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.6841 - accuracy: 0.7946 - val_loss: 1.2807 - val_accuracy: 0.7315
Epoch 16/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.6796 - accuracy: 0.7981 - val_loss: 1.2263 - val_accuracy: 0.7429
Epoch 17/40
1000/1000 [==============================] - 597s 597ms/step - loss: 0.6579 - accuracy: 0.8032 - val_loss: 1.2329 - val_accuracy: 0.7394
Epoch 18/40
1000/1000 [==============================] - 596s 596ms/step - loss: 0.6496 - accuracy: 0.8075 - val_loss: 1.2351 - val_accuracy: 0.7460
```

In [ ]:
```
plot_accuracy(history_moderate_effB0)
```



Summary: (EfficientNet B0)

> Model metrics: Model name: 32,000
>
> > Highest training accuracy : 0.8075
> >
> > Lowest training loss: 0.6496
> >
> > Highest validation accuracy: 0.7460
> >
> > Lowest validation loss: 1.2351

ResNet50

> Fit the model for given dataset (32,000)

In [ ]:
```
history_moderate_resnet50 = small_resnet50_model.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
1000/1000 [==============================] - 634s 619ms/step - loss: 1.6593 - accuracy: 0.5274 - val_loss: 1.1839 - val_accuracy: 0.6441
Epoch 2/40
1000/1000 [==============================] - 615s 615ms/step - loss: 1.1578 - accuracy: 0.6489 - val_loss: 1.0686 - val_accuracy: 0.6847
Epoch 3/40
1000/1000 [==============================] - 616s 616ms/step - loss: 1.0440 - accuracy: 0.6873 - val_loss: 1.0489 - val_accuracy: 0.6949
Epoch 4/40
1000/1000 [==============================] - 616s 616ms/step - loss: 0.9789 - accuracy: 0.7041 - val_loss: 1.0781 - val_accuracy: 0.6804
Epoch 5/40
1000/1000 [==============================] - 616s 616ms/step - loss: 0.9299 - accuracy: 0.7217 - val_loss: 1.1660 - val_accuracy: 0.6683
Epoch 6/40
1000/1000 [==============================] - 615s 615ms/step - loss: 0.8883 - accuracy: 0.7325 - val_loss: 1.0003 - val_accuracy: 0.7138
Epoch 7/40
1000/1000 [==============================] - 616s 616ms/step - loss: 0.8560 - accuracy: 0.7464 - val_loss: 1.0045 - val_accuracy: 0.7169
Epoch 8/40
1000/1000 [==============================] - 616s 616ms/step - loss: 0.8295 - accuracy: 0.7532 - val_loss: 1.0649 - val_accuracy: 0.7082
Epoch 9/40
1000/1000 [==============================] - 617s 616ms/step - loss: 0.8024 - accuracy: 0.7606 - val_loss: 1.0830 - val_accuracy: 0.7135
Epoch 10/40
1000/1000 [==============================] - 615s 615ms/step - loss: 0.7760 - accuracy: 0.7694 - val_loss: 1.0477 - val_accuracy: 0.7390
Epoch 11/40
1000/1000 [==============================] - 616s 616ms/step - loss: 0.7475 - accuracy: 0.7794 - val_loss: 0.9752 - val_accuracy: 0.7452
Epoch 12/40
1000/1000 [==============================] - 615s 615ms/step - loss: 0.7294 - accuracy: 0.7828 - val_loss: 1.1078 - val_accuracy: 0.7185
Epoch 13/40
1000/1000 [==============================] - 615s 615ms/step - loss: 0.7087 - accuracy: 0.7909 - val_loss: 1.0795 - val_accuracy: 0.7204
Epoch 14/40
1000/1000 [==============================] - 615s 615ms/step - loss: 0.6973 - accuracy: 0.7962 - val_loss: 1.1472 - val_accuracy: 0.7099
Epoch 15/40
1000/1000 [==============================] - 615s 615ms/step - loss: 0.6794 - accuracy: 0.8005 - val_loss: 1.1071 - val_accuracy: 0.7263
```

Summary: (ResNet50)

> Model metrics: Model name: 32,000
>
> > Highest training accuracy : 0.8005
> >
> > Lowest training loss: 0.6794
> >
> > Highest validation accuracy: 0.7452
> >
> > Lowest validation loss: 0.9752

>

fit model with dataset size: 160,000 train and 32,000 validation images

In [ ]:
```
# Generate train dataset for 1000x768 with 1000 images for each category
source_path = "/Volumes/T7/rvl-cdip/train"
target_path = "/Volumes/T7/sample_10000/rvl-cdip/"
desired_size_per_category = 10000
width = 1000
height = 768

generate_dataset(width, height, target_path, source_path, desired_size_per_category)
```

```
Directory /Volumes/T7/rvl-cdip/train/15 has 19975 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/1 has 19957 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/7 has 19997 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/10 has 20010 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/3 has 20031 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/14 has 20006 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/11 has 19944 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/13 has 20042 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/0 has 20103 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/2 has 19954 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/9 has 19987 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/4 has 19963 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/12 has 20043 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/5 has 19984 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/6 has 19829 files, we have to choose 10000 files (randomly).
Directory /Volumes/T7/rvl-cdip/train/8 has 20012 files, we have to choose 10000 files (randomly).
```

In [ ]:
```
# Generate test dataset for 1000x768 with 1000 images for each category

source_path = "/Volumes/T7/rvl-cdip/test"
target_path = "/Volumes/T7/sample_10000/rvl-cdip/"
desired_size_per_category = 2000
width = 1000
height = 768

generate_dataset(width, height, target_path, source_path, desired_size_per_category)
```

```
Directory /Volumes/T7/rvl-cdip/test/3 has 2532 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/14 has 2536 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/2 has 2516 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/9 has 2463 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/6 has 2570 files, we have to choose 2000 files (randomly).
TiffPages: invalid page offset 85476
Directory /Volumes/T7/rvl-cdip/test/13 has 2435 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/8 has 2527 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/0 has 2464 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/7 has 2472 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/5 has 2498 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/1 has 2506 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/15 has 2492 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/11 has 2477 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/10 has 2505 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/12 has 2489 files, we have to choose 2000 files (randomly).
Directory /Volumes/T7/rvl-cdip/test/4 has 2515 files, we have to choose 2000 files (randomly).
```

In [13]:
```
datagen = kimage.ImageDataGenerator()

# load and iterate training dataset
train_it = datagen.flow_from_directory('/content/sample_10000/rvl-cdip/train/', class_mode='categorical', target_size=(1024, 768), color_mode='grayscale')
# load and iterate test datase
test_it = datagen.flow_from_directory('/content/sample_10000/rvl-cdip/test/', class_mode='categorical',  target_size=(1024, 768), color_mode='grayscale')
```

```
Found 159991 images belonging to 16 classes.
Found 31985 images belonging to 16 classes.
```

In [27]:
```
# Early stopping if no improvement noticed
callbacks_list = [
        keras.callbacks.EarlyStopping(
            monitor = 'val_accuracy', # Use accuracy to monitor the model
            patience = 4 # Stop after one step with lower accuracy
        )
]
```

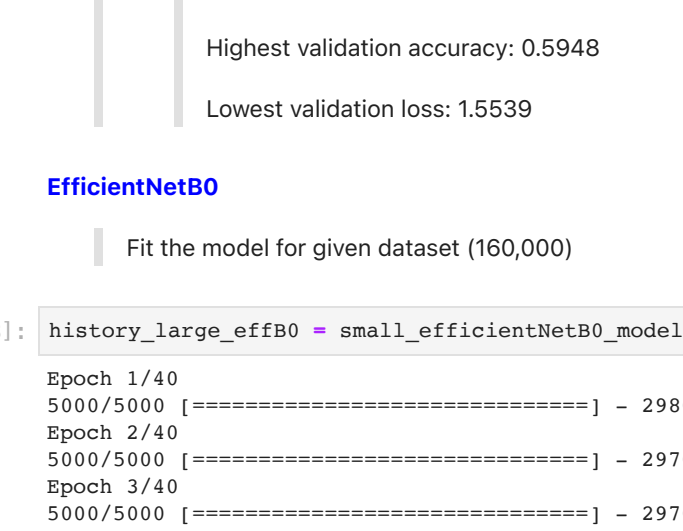
CNN

> Fit the model for given dataset (160,000)

In [10]:
```
large_model = nn.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
5000/5000 [==============================] - 1700s 338ms/step - loss: 2.0067 - accuracy: 0.4202 - val_loss: 1.6538 - val_accuracy: 0.5393
Epoch 2/40
5000/5000 [==============================] - 1626s 325ms/step - loss: 1.7410 - accuracy: 0.5113 - val_loss: 1.6505 - val_accuracy: 0.5455
Epoch 3/40
5000/5000 [==============================] - 1621s 324ms/step - loss: 1.7017 - accuracy: 0.5296 - val_loss: 1.5355 - val_accuracy: 0.5781
Epoch 4/40
5000/5000 [==============================] - 1625s 325ms/step - loss: 1.6860 - accuracy: 0.5403 - val_loss: 1.5539 - val_accuracy: 0.5948
Epoch 5/40
5000/5000 [==============================] - 1629s 326ms/step - loss: 1.6769 - accuracy: 0.5489 - val_loss: 1.6582 - val_accuracy: 0.5531
Epoch 6/40
5000/5000 [==============================] - 1622s 324ms/step - loss: 1.6665 - accuracy: 0.5560 - val_loss: 1.5796 - val_accuracy: 0.5841
Epoch 7/40
5000/5000 [==============================] - 1622s 324ms/step - loss: 1.6774 - accuracy: 0.5611 - val_loss: 1.6959 - val_accuracy: 0.5597
Epoch 8/40
5000/5000 [==============================] - 1624s 325ms/step - loss: 1.6902 - accuracy: 0.5621 - val_loss: 1.5908 - val_accuracy: 0.5914
```

In [11]:
```
plot_accuracy(large_model)
```



Summary: (CNN)

> Model metrics: Model name: 160,000
>
> > Highest training accuracy : 0.5621
> >
> > Lowest training loss: 1.6774
> >
> > Highest validation accuracy: 0.5948
> >
> > Lowest validation loss: 1.5539

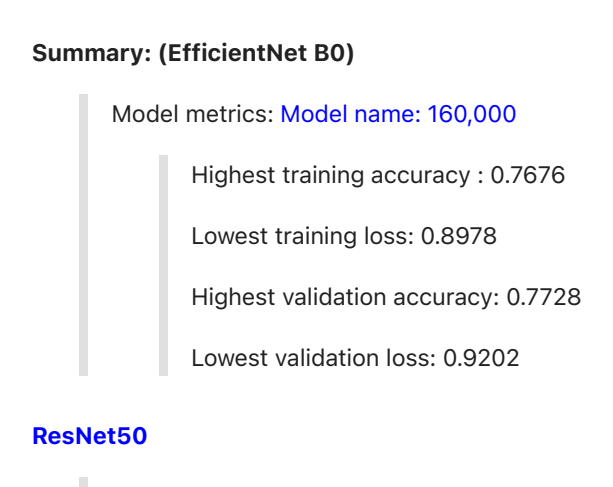
EfficientNetB0

> Fit the model for given dataset (160,000)

In [28]:
```
history_large_effB0 = small_efficientNetB0_model.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
5000/5000 [==============================] - 2980s 595ms/step - loss: 1.1677 - accuracy: 0.6463 - val_loss: 0.9706 - val_accuracy: 0.7213
Epoch 2/40
5000/5000 [==============================] - 2970s 594ms/step - loss: 0.9785 - accuracy: 0.7140 - val_loss: 0.9744 - val_accuracy: 0.7247
Epoch 3/40
5000/5000 [==============================] - 2970s 594ms/step - loss: 0.9337 - accuracy: 0.7321 - val_loss: 0.9424 - val_accuracy: 0.7385
Epoch 4/40
5000/5000 [==============================] - 2970s 594ms/step - loss: 0.9115 - accuracy: 0.7443 - val_loss: 0.9428 - val_accuracy: 0.7503
Epoch 5/40
5000/5000 [==============================] - 2971s 594ms/step - loss: 0.9010 - accuracy: 0.7508 - val_loss: 0.9478 - val_accuracy: 0.7504
Epoch 6/40
5000/5000 [==============================] - 2970s 594ms/step - loss: 0.8936 - accuracy: 0.7559 - val_loss: 0.9727 - val_accuracy: 0.7581
Epoch 7/40
5000/5000 [==============================] - 2973s 595ms/step - loss: 0.8978 - accuracy: 0.7574 - val_loss: 0.9202 - val_accuracy: 0.7694
Epoch 8/40
5000/5000 [==============================] - 2974s 595ms/step - loss: 0.8924 - accuracy: 0.7613 - val_loss: 0.9810 - val_accuracy: 0.7635
Epoch 9/40
5000/5000 [==============================] - 2974s 595ms/step - loss: 0.8988 - accuracy: 0.7626 - val_loss: 0.9437 - val_accuracy: 0.7728
Epoch 10/40
5000/5000 [==============================] - 2971s 594ms/step - loss: 0.8985 - accuracy: 0.7654 - val_loss: 1.0215 - val_accuracy: 0.7658
Epoch 11/40
5000/5000 [==============================] - 2971s 594ms/step - loss: 0.9078 - accuracy: 0.7642 - val_loss: 1.0122 - val_accuracy: 0.7632
Epoch 12/40
5000/5000 [==============================] - 2976s 595ms/step - loss: 0.9146 - accuracy: 0.7650 - val_loss: 1.0053 - val_accuracy: 0.7602
Epoch 13/40
5000/5000 [==============================] - 2975s 595ms/step - loss: 0.9176 - accuracy: 0.7676 - val_loss: 1.0159 - val_accuracy: 0.7610
```

In [29]:
```
plot_accuracy(history_large_effB0)
```



Summary: (EfficientNet B0)

> Model metrics: Model name: 160,000
>
> > Highest training accuracy : 0.7676
> >
> > Lowest training loss: 0.8978
> >
> > Highest validation accuracy: 0.7728
> >
> > Lowest validation loss: 0.9202

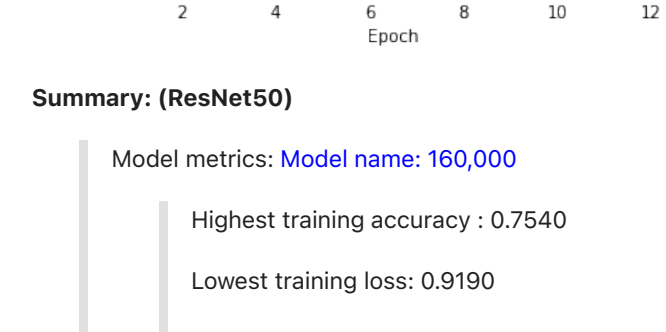
ResNet50

> Fit the model for given dataset (160,000)

In [14]:
```
history_large_resnet50 = small_resnet50_model.fit(train_it, epochs=40, validation_data=test_it, callbacks = callbacks_list)
```

```
Epoch 1/40
5000/5000 [==============================] - 3123s 622ms/step - loss: 1.2210 - accuracy: 0.6380 - val_loss: 1.0788 - val_accuracy: 0.6798
Epoch 2/40
5000/5000 [==============================] - 3058s 612ms/step - loss: 1.0010 - accuracy: 0.7089 - val_loss: 1.0181 - val_accuracy: 0.7089
Epoch 3/40
5000/5000 [==============================] - 3059s 612ms/step - loss: 0.9606 - accuracy: 0.7258 - val_loss: 0.9870 - val_accuracy: 0.7200
Epoch 4/40
5000/5000 [==============================] - 3058s 612ms/step - loss: 0.9428 - accuracy: 0.7344 - val_loss: 0.9965 - val_accuracy: 0.7175
Epoch 5/40
5000/5000 [==============================] - 3058s 612ms/step - loss: 0.9299 - accuracy: 0.7402 - val_loss: 0.9938 - val_accuracy: 0.7293
Epoch 6/40
5000/5000 [==============================] - 3059s 612ms/step - loss: 0.9264 - accuracy: 0.7423 - val_loss: 0.9427 - val_accuracy: 0.7411
Epoch 7/40
5000/5000 [==============================] - 3056s 611ms/step - loss: 0.9181 - accuracy: 0.7460 - val_loss: 0.9984 - val_accuracy: 0.7343
Epoch 8/40
5000/5000 [==============================] - 3061s 612ms/step - loss: 0.9198 - accuracy: 0.7485 - val_loss: 0.9545 - val_accuracy: 0.7441
Epoch 9/40
5000/5000 [==============================] - 3056s 611ms/step - loss: 0.9222 - accuracy: 0.7487 - val_loss: 1.0831 - val_accuracy: 0.7192
Epoch 10/40
5000/5000 [==============================] - 3057s 611ms/step - loss: 0.9207 - accuracy: 0.7506 - val_loss: 1.0048 - val_accuracy: 0.7426
Epoch 11/40
5000/5000 [==============================] - 3056s 611ms/step - loss: 0.9136 - accuracy: 0.7519 - val_loss: 1.0520 - val_accuracy: 0.7366
Epoch 12/40
5000/5000 [==============================] - 3055s 611ms/step - loss: 0.9190 - accuracy: 0.7540 - val_loss: 1.0841 - val_accuracy: 0.7262
```

In [15]:
```
plot_accuracy(history_large_resnet50)
```

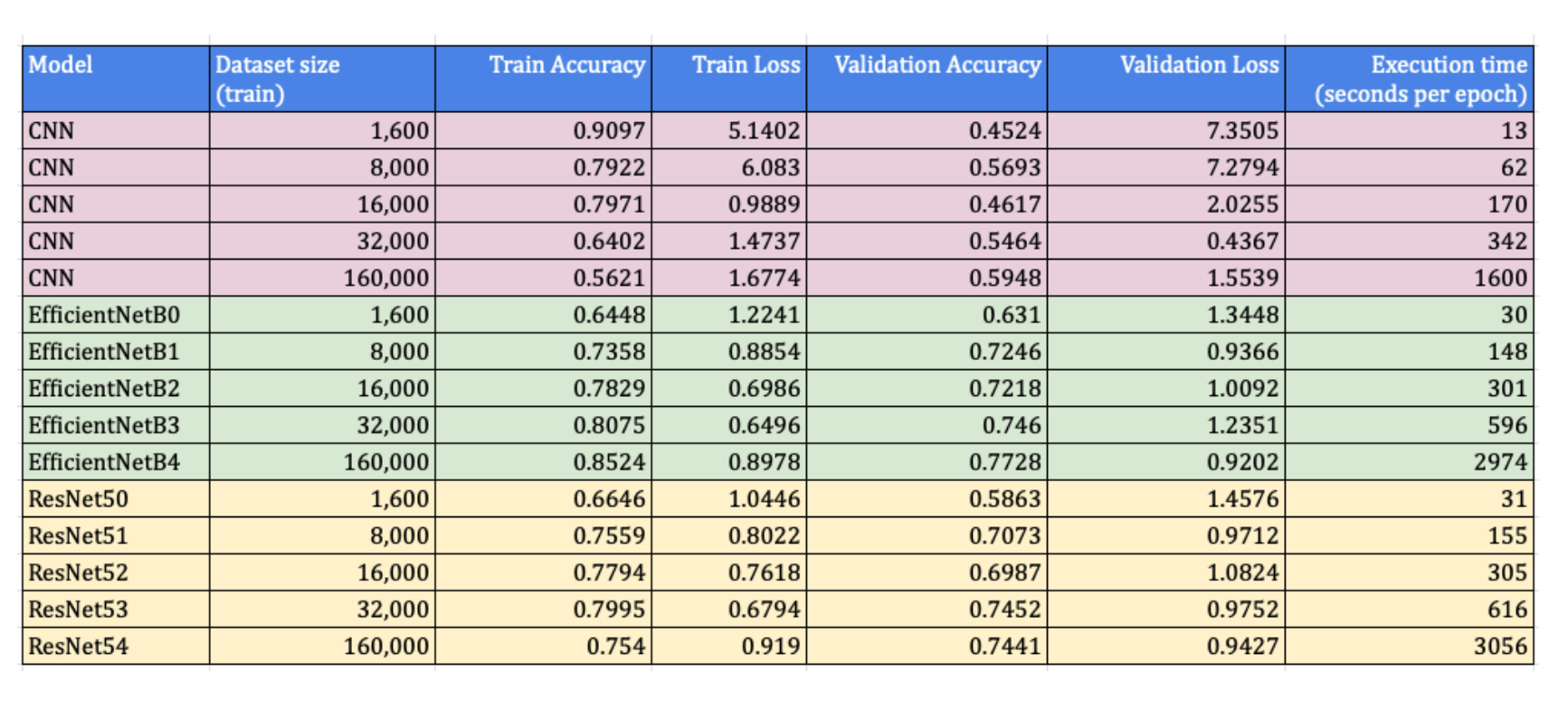


Summary: (ResNet50)

> Model metrics: Model name: 160,000
>
> > Highest training accuracy : 0.7540
> >
> > Lowest training loss: 0.9190
> >
> > Highest validation accuracy: 0.7441
> >
> > Lowest validation loss: 0.9427

Conclusion

> I fit three different models: convolution neural network, transfer learning based on EfficientNetB0, and transfer learning based on ResNet50 for five different datasets.
>
> I have collected following metrics for each of them as below:

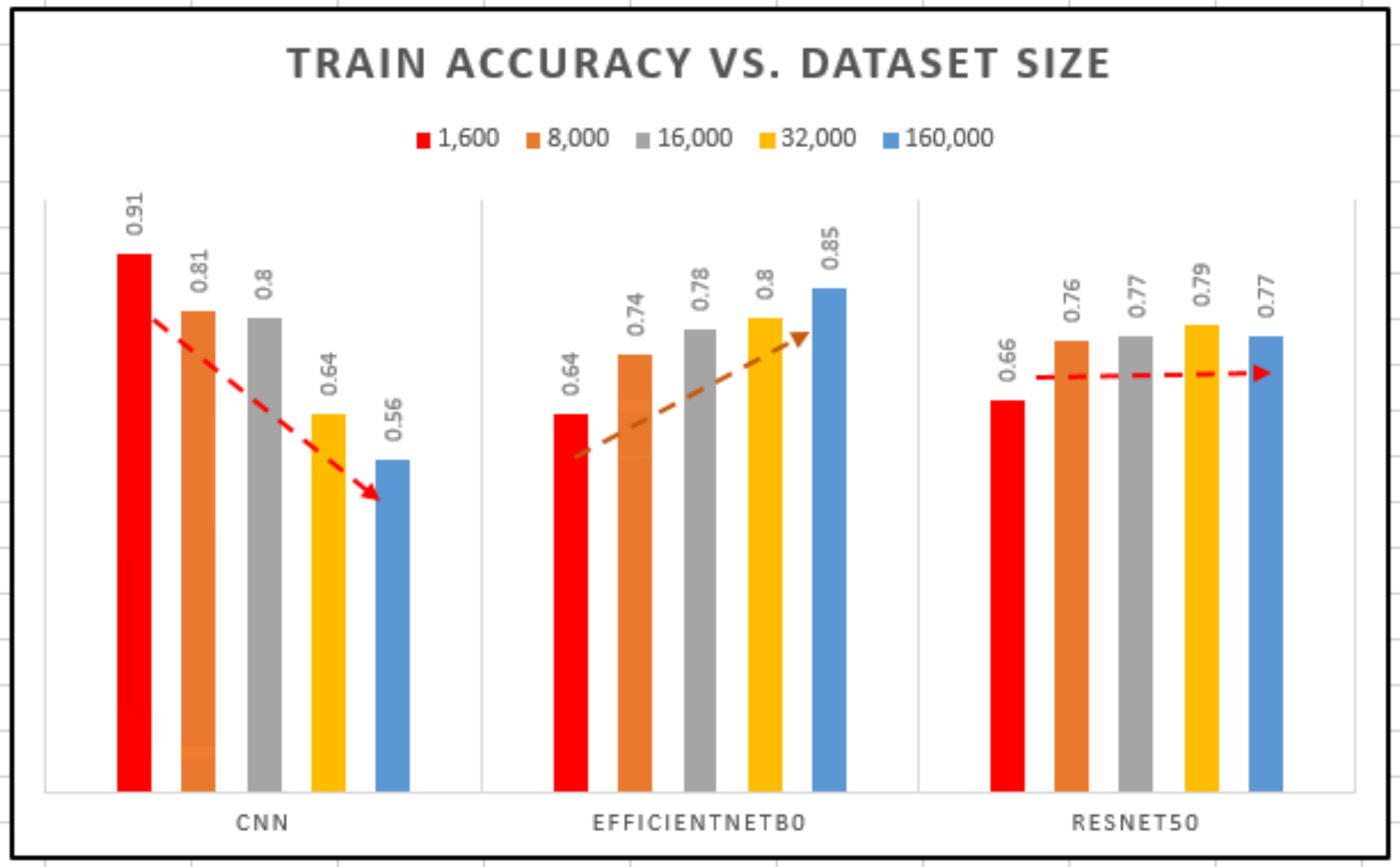


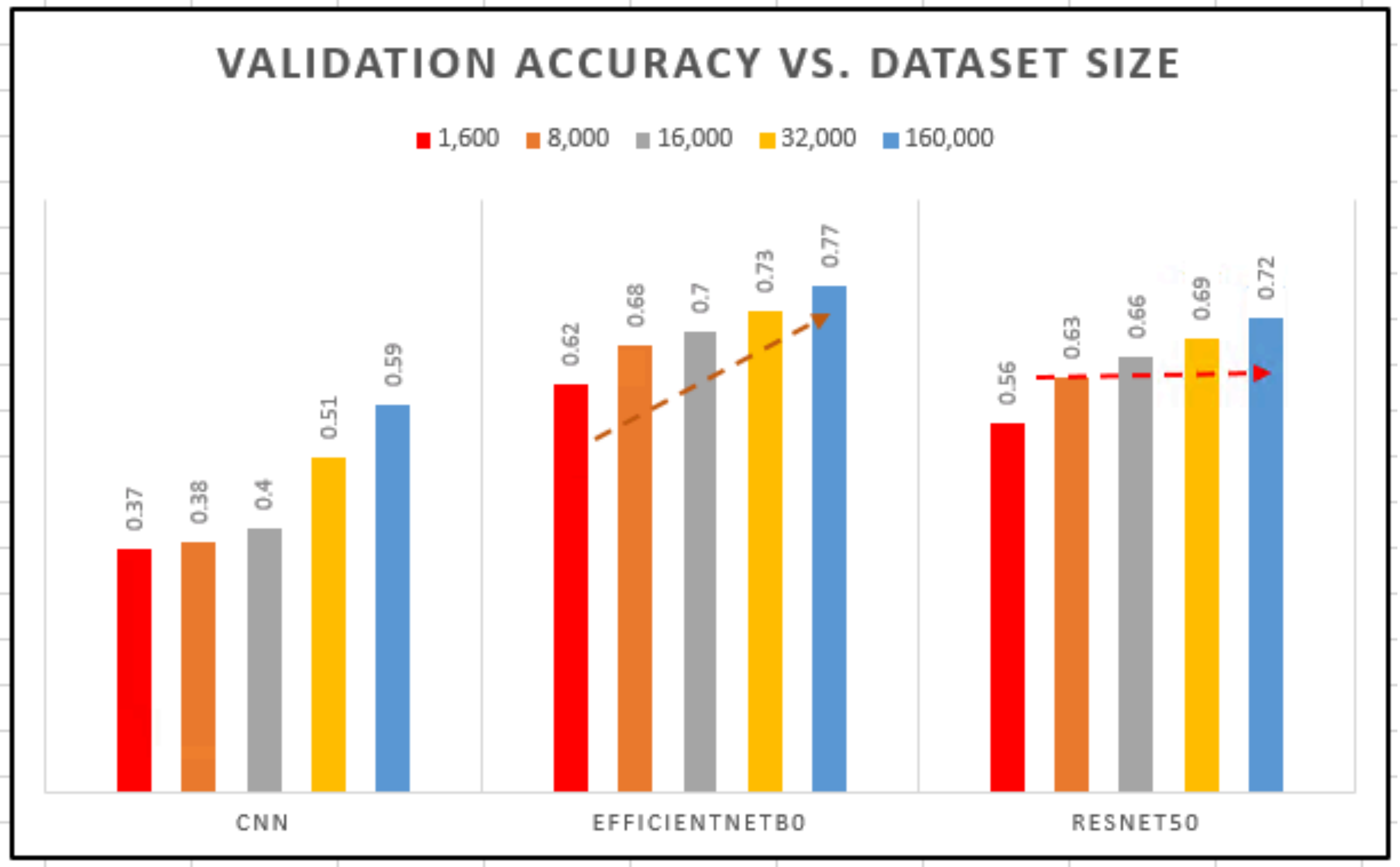
Model	Dataset size (train)	Train Accuracy	Train Loss	Validation Accuracy	Validation Loss	Execution time (seconds per epoch)
CNN	1,600	0.9097	5.1402	0.4524	7.3505	13
CNN	8,000	0.7922	6.083	0.5693	7.2794	62
CNN	16,000	0.7971	0.9889	0.4617	2.0255	170
CNN	32,000	0.6402	1.4737	0.5464	0.4367	342
CNN	160,000	0.5621	1.6774	0.5948	1.5539	1600
EfficientNetB0	1,600	0.6448	1.2241	0.631	1.3448	30
EfficientNetB1	8,000	0.7358	0.8854	0.7246	0.9366	148
EfficientNetB2	16,000	0.7829	0.6986	0.7218	1.0092	301
EfficientNetB3	32,000	0.8075	0.6496	0.746	1.2351	596
EfficientNetB4	160,000	0.8524	0.8978	0.7728	0.9202	2974
ResNet50	1,600	0.6646	1.0446	0.5863	1.4576	31
ResNet51	8,000	0.7559	0.8022	0.7073	0.9712	155
ResNet52	16,000	0.7794	0.7618	0.6987	1.0824	305
ResNet53	32,000	0.7995	0.6794	0.7452	0.9752	616
ResNet54	160,000	0.754	0.919	0.7441	0.9427	3056

Observations:

> Accuracy
>
> > CNN:
> >
> > > The model shows overfitting problem with smaller dataset as traing accuracy is very high compare to validation accurancy. The model addresses overfitting problem by adjusting weights with larget dataset. Though, the validation accuracy with largest dataset is just 56%. The model shows lack of capacity of classifier.
> >
> > EfficientNetB0:

> > > The model shows improvement as dataset size increases. The model does not indicate any overfitting problem as train and validation accuracy are comparable for all datasets. This model is strong candidate for generalization.
> >
> > ResNet50:
> >
> > > The model based on ResNet50 does not indicate overfitting problem. It is candidate for generalizaiton.
> >
> > **Conclusion: Comparing accuracy among models, we can say that EfficientNet and ResNet based models can be genearalized. CNN model improves with dataset but produces sub-optimal validation accuracy compare to other models.**
>
> Loss
>
> > CNN:
> >
> > > The model suffers from overfitting problem as loss values are high for train and validation. The loss function decreasing with dataset size which indicates limited training data as well as lack of complexity in the model. The model with larget dataset has high loss function, indicates it is not suitable for generalization.
> >
> > EfficientNetB0:
> >
> > > The model has comparable loss function between train and validation. The loss function improves with dataset size. The model is candidate for generalization.
> >
> > ResNet50:
> >
> > > The model based on ResNet50 has comparable loss function between train and validation, though the difference is higher than efficientNet based model. The model can be generalized as no significant issue noticed.
> >
> > **Conclusion: Comparing loss functions among models, we can say that EfficientNet based model is stronger candidate to be genearalized. CNN model improves with dataset but produces sub-optimal loss function compare to other models.**
>
> The model based on EfficientNetB0 gives best results for given datasets. Below diagram, side by side comparision, also confirms EfficientNetB0 as better model.
>
> **Important to note:**
>
> > 1. The conclusion is based on given dataset and steps performed.
> > 2. There are few next steps to improve the performance like segementation of image to have multiple small models addressing Header, Footer and Body of the image.
> > 3. Epochs: Above results indicates that early stopping callback stops evaluation under 20 Epochs. Due to computation limitation and increasing exeuction time, I used patience of 4 epochs. I believe, more epochs would allow convergence of train and test accuracy, specifically for EfficientNet and ResNet models.





Reference:

> 1. Ref: A. W. Harley, A. Ufkes, K. G. Derpanis, "Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval," in ICDAR, 2015 https://www.cs.cmu.edu/~aharley/icdar15
> 2. Data exploration https://medium.com/analytics-vidhya/rvl-cdip-ryerson-vision-lab-complex-document-information-processing-aa30b00a2b1e
> 3. Convolution network https://www.researchgate.net/figure/Sample-images-from-the-RVL-CDIP-dataset-One-image-from-each-class-is-depicted-From-left_fig3_316073094
> 4. Transfer learning https://keras.io/api/applications/
> 5. **Class Notes** CSCI E-25 Computer Vision by Stephen Elston PhD, Principal Consultant, Quantia Analytics LLC